山东大学 计算机科学与技术 学院

数据结构与算法 课程实验报告

学号：202200130077	姓名：薛润泽	班级：人工智能
实验题目：实验 5 数组和矩阵		
实验学时：2	实验日期： 2023.10.18	
实验目的： 掌握稀疏矩阵结构的描述及操作的实现。		
软件开发工具： DEV C++		

1. 实验内容

创建稀疏矩阵类（参照课本 MatrixTerm 三元组定义），采用行主顺序把稀疏矩阵非 0 元素映射到一维数组中，实现操作：两个稀疏矩阵相加、两个稀疏矩阵相乘、稀疏矩阵的转置、输出矩阵。

2. 数据结构与算法描述 （整体思路描述，所需要的数据结构与算法）

数据结构与算法：使用三元组在一维数组中构建系数矩阵，稀疏矩阵乘法

思路描述：

稀疏矩阵的难点在于实现线性复杂度的矩阵转置和复杂度为矩阵中非零元素个数三次方的矩阵乘法。

对于实现线性复杂度的矩阵转置操作，通过遍历一维数组，记录数组中每一列的元素个数，通过每一列的元素个数建立一个索引数组，记录每一列的元素转置后对应数组中对应矩阵相应行的第一个元素。并通过该记录来转置数组。具体操作为遍历数组，并查看每个元素的列号，通过该列号找到在新矩阵中对应的索引，将该元素赋值到该索引处，并将索引+1。重复该操作直到遍历完整个矩阵。

对于实现矩阵乘法操作，考虑普通矩阵乘法操作，通常要一行乘一列，算完结果矩阵中的对应元素，再进行一行乘一列的运算。为了提高效率，希望实现左矩阵的索引不减的操作，申请一个一维临时矩阵，矩阵元素个数等于该稀疏矩阵的列数+1，该矩阵用来记录左矩阵中的一行元素分别与右矩阵中的每个对应元素的乘积的和。具体描述，以左矩阵第一行第一列的元素为例子，这个元素分别与右矩阵对应第一行的每个元素做乘积，结果加到临时矩阵，左矩阵再遍历到第一行第二个元素，该元素再与右矩阵中第二行的相应元素做乘积，结果加到临时矩阵，依次循环，直到完成左矩阵一行的遍历，将临时矩阵的非零元素加入到结果矩阵中，完成左矩阵第一行与右矩阵每一列的乘积。

函数和类描述：

（1） int Read();函数用来快速读入输入，降低读入操作所用时间。

（2） struct matrixTerm;该结构体用来记录三元组，该元素的值，该元素的行、列

（3） void spareMatrix:: resetMatrix();用来重置矩阵。delete 原来的 element 数组，并重新申请一个行数为 n，列数为 m 的一维数组。同时更新原矩阵的行列数。输入新矩阵，完成对矩阵的更新。

（4） class iterator;稀疏矩阵中的迭代器类，用于矩阵加法的元素遍历。

（5） void MatrixMultiply(spareMatrix*&A, spareMatrix*&B);该函数用来计算矩阵乘法。首先记录右矩阵的每行元素个数，通过该记录计算出 B 矩阵每行首个元素的索引。计算矩阵乘法，分三重循环，第一重循环遍历左矩阵的每一行，第二重循环用来遍历左矩阵每一行中的每个元素，第三重循环用来计算左矩阵中每个元素与右矩阵中某行元素依次相乘，得到的结果加和在临时矩阵中。第一重循环循环一次后，把记录在临时矩阵中的非零元素插入结果矩阵。并将临时矩阵归零。直到左矩阵遍历完后结束三重循环。最终得到矩阵乘法结果。

（6） void MatrixPlus(spareMatrix*& A,spareMatrix*& B);该函数用来计算矩阵加法。通过使用迭代器，遍历 A，B 两个矩阵，如果两个矩阵在对应位置有一个矩阵存在非零元素或者两个矩

阵都存在非零元素，则进行加法。并将非零结果加入到结果矩阵中。

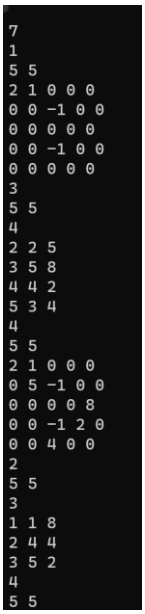
（7） void MatrixInverse();该函数实现矩阵转置操作。通过遍历一维数组，记录数组中每一列的元素个数，通过每一列的元素个数建立一个索引数组，记录每一列的元素转置后对应数组中对应矩阵相应行的第一个元素。并通过该记录来转置数组。具体操作为遍历数组，并查看每个元素的列号，通过该列号找到在新矩阵中对应的索引，将该元素赋值到该索引处，并将索引+1。重复该操作直到遍历完整个矩阵。

3. 测试结果（测试输入，测试输出）

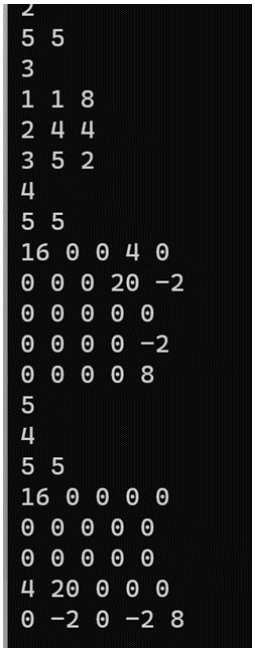
测试输入

```
7
1
5 5
2 1 0 0 0
0 0 -1 0 0
0 0 0 0 0
0 0 -1 0 0
0 0 0 0 0
3
5 5
4
2 2 5
3 5 8
4 4 2
5 3 4
4
2
5 5
3
1 1 8
2 4 4
3 5 2
4
5
4
```

测试输出图 1



测试输出图 2（续）



4. 分析与探讨（结果分析，若存在问题，探讨解决问题的途径）

最开始写矩阵转置和矩阵乘法时发生了 TLE 的情况，初步判断是复杂度过高导致的。矩阵转置我最开始采用的是每个元素互换行列号，然后再对整个数组行优先排序，复杂度过高。优化矩阵转置后可以实现线性复杂度。最开始写矩阵乘法是通过使用迭代器遍历两个矩阵，通过正常的矩阵乘法计算方法来计算，复杂度过高。通过改进的矩阵乘法操作，降低了复杂度。

5. 附录：实现源代码（本实验的全部源程序代码，程序风格清晰易理解，有充分的注释）

```
#include<iostream>
int Read(){//快速读入
    int x = 0, f = 1;
    char c = getchar();
    while(c < '0' || c > '9'){
        if(c == '-'){
            f = -1;
        }
        c = getchar();
    }
    while(c >= '0' && c <= '9'){
        x = x*10+c-'0';
        c = getchar();
    }
    return x*f;
}

struct matrixTerm{//用于存储稀疏矩阵中每个元素值以及所在的行，列
    int row = -1;//判断未被初始化
```

```
    int col = -1;
    int value;
    matrixTerm& operator=(const matrixTerm& A){//重载运算符 =
        this->row = A.row;
        this->col = A.col;
        this->value = A.value;
        return *this;
    }
};

class spareMatrix{
public:
    void resetMatrix(){//重置数组
        int n, m;
        realnum = 0;
        n = Read(); m = Read();
        delete []element;
        element = new matrixTerm[n*m + 1];//最多会占据 n*m + 1 个位置
        for(int i = 1; i<= n*m; i++){       //初始化 element 数组行为-1
            element[i].row = -1;
        }
        rows = n; cols = m;
        int cnt = 1;//记录数组的索引
        int cinvalue;
        for(int i = 1; i<= n; i++){
            for(int j = 1; j <=m; j++){
                cinvalue = Read();
                if(cinvalue != 0){
                    realnum++;
                    element[cnt].row = i;
                    element[cnt].col = j;
                    element[cnt].value = cinvalue;
                    cnt++;
                }
            }
        }
    }

    ~spareMatrix(){delete []element;}

    int getrows(){return rows;}
    matrixTerm* getElement(){return (element + 1);}
    int getcols(){return cols;}

    class iterator{
    public:
```

```
        matrixTerm* ptr;
        iterator(){};//空构造函数
        void get(matrixTerm* temp){
            ptr = temp;
        }
        iterator & operator ++(int){
            iterator* temp    =this;
            ptr++;
            return *temp;
        }
        iterator& operator++(){
            ptr++;
            return *this;
        }
        matrixTerm &operator*()const{
            return *ptr;
        }
    };
void MatrixMultiply(spareMatrix*&A, spareMatrix*&B){
    if(A->getcols()!=B->getrows()){//如果两个矩阵不满足矩阵乘法
        delete A;
        *this = *B;
        printf("-1\n");
        return;
    }   //两个矩阵满足矩阵乘法
    rows = A->getrows();
    cols = B->getcols();
    element = new matrixTerm[rows * cols + 1];//结果矩阵
    for(int i    = 1; i <= rows * cols; i++){
        element[i].row = -1;                    //初始化结果矩阵元素行为-1
    }
    int *rowSize = new int[B->getrows() + 1]; //记录 B 矩阵每行的元素个数
    int *rowNext = new int[B->getrows() + 1];//记录 B 矩阵每行的起始索引
    for(int i = 1; i <= B->getrows(); i++){
        rowSize[i] = 0;                             //初始化起始 rowSize
    }
    for(int i = 1; i <= B->getrows() *B->getcols(); i++){
        if(B->element[i].row < 1 || B->element[i].row > B->getrows()) break;//元素判断为无
效，则结束
        rowSize[B->element[i].row]++;//元素有效，则在对应记录行元素个数的 rowSize 处
++
    }
    rowNext[1] = 1;//初始化第一行第一个元素索引为 1
    for(int i = 2; i <= B->getrows(); i++){
        rowNext[i] = rowNext[i - 1] + rowSize[i - 1];//计算每行索引
    }
```

```
    int ans[505];//记录遍历完左侧矩阵一行时的结果
    int count = 0;
    for(int i = 1; i <= A->getrows() * A->getcols(); i++){
        if(A->element[i].row < 1 || A->element[i].row >A->getrows()) break;
        count++;                                                    //记录稀疏矩阵 A 的非零元素个
数
    }
    int cnt = 1; //记录 element 矩阵索引
    int p = 1;//A 矩阵索引，初始化为 1
    for(int i = 1; i <= A->getrows() && p <= count; i++){//p 是 A 矩阵的一个索引, i 记录 A 矩
阵现在元素的行数
        for(int clearans = 1; clearans<505; clearans++) ans[clearans] = 0 ;//清空 ans 数组
        while(p <= count && A->element[p].row == i){//当 p 没有超过总个数且 p 的行数是 i
时
            int t = A->element[p].col; //t  记录该元素在 A 矩阵中的列数
            if(rowSize[t] != 0){//如果 B 矩阵中，对应行元素个数为 0，则结果直接为 0
                for(int q = rowNext[t];q < rowNext[t] + rowSize[t]; q++){//q 定位到 B 矩阵相
应的起始行索引
                ans[B->element[q].col] += A->element[p].value * B->element[q].value;
                 }
            }
            p++;
           }
           for(int search = 1; search <= B->getrows(); search++){//遍历结果矩阵，将不为 0
的结果输入到矩阵中
           if(ans[search]!=0) {
               element[cnt].row = i;
               element[cnt].col = search;
               element[cnt].value = ans[search];
               cnt++;
           }
        }
    }
    delete A; delete B;
}

void MatrixPlus(spareMatrix*& A, spareMatrix*& B){
    if(A->getrows() != B->getrows() || A->getcols() != B->getcols()){//如果 A,B 的行列不相等
           delete A;                            //释放 A 的空间
           *this = *B;                          //直接让 this 等于指针 B
           printf("-1\n");              //不合法输出-1
           return ;
        }
    rows = A->getrows();//A，B 行列相等
    cols = A->getcols();
    element = new matrixTerm[rows * cols + 1];
```

```
    for(int i = 1; i <= rows*cols; i++){
        element[i].row = -1;
    }
    spareMatrix::iterator ptrA; spareMatrix::iterator ptrB;//使用迭代器来遍历矩阵
    ptrA.get(A->getElement()); ptrB.get(B->getElement());
    int cnt = 1;//记录结果矩阵索引
    for(int i = 1; i <= rows; i++){
        for(int j = 1; j <= cols; j++){
            while((*(ptrA)).row != -1 && (*(ptrA)).row < i) ptrA++;                    //
如果该元素仍然是有效的并且行小于要求的行，则++看到下一个元素
            while((*(ptrA)).row != -1 && (*(ptrA)).row == i &&((*(ptrA)).col) < j)ptrA++ ;//
如果元素有效，行是要求行，但是列小于要求的列，则++看到下一个元素
            while((*(ptrB)).row != -1 && (*(ptrB)).row < i) ptrB++;
            while((*(ptrB)).row != -1 && (*(ptrB)).row == i &&((*(ptrB)).col) < j)ptrB++ ;//此
时 ptrA，ptrB 位于目标位置或更前方位置
            if(((*(ptrA)).row == i && (*(ptrA)).col == j) ||((*(ptrB)).row == i && (*(ptrB)).col
== j)){//如果两个矩阵在该 i，j 位置任意有一个有非零元素
                int temp = 0, tempA = 0, tempB = 0;
                if((*(ptrA)).row == i && (*(ptrA)).col == j) tempA = (*(ptrA)).value;     //判
断 A 在该位置是否有非零元素
                if((*(ptrB)).row == i && (*(ptrB)).col == j) tempB = (*(ptrB)).value;     //判断
B 在该位置是否有非零元素
                temp = tempA+ tempB;
                if(temp != 0){//如果两个数加和不为零，则加入结果矩阵
                    realnum++;
                    element[cnt].row = i;
                    element[cnt].col = j;
                    element[cnt].value = temp;
                    cnt++;
                }
            }
        }
    }
    delete A;delete B;
}

void MatrixInverse(){ //矩阵转置
    realnum = 0;
    for(int i =1; i<=rows*cols; i++){
        if(element[i].row >rows|| element[i].row < 1) break;
        realnum ++;    //重新计算矩阵中非零元素个数
    }

    int *colSize = new int[cols + 1];   //记录未转置前记录列的元素个数
    int *rowNext = new int[cols + 1];  //记录转置后每一行的第一个元素索引
    for(int i = 1; i <= cols ; i++){
```

```
        colSize[i] = 0;                    //初始化 colSize 为 0
    }
    for(int i = 1; i <= realnum; i++){
        colSize[element[i].col] ++;        //计算 colSize
    }
    rowNext[1] = 1;                        //初始化 rowNext[1] = 1  即转置后第一行的第一个
元素索引为 1
    for(int i = 2; i <= cols; i++){
        rowNext[i] = rowNext[i - 1] + colSize[i - 1];
    }
    matrixTerm* temp = new matrixTerm[cols * rows + 1]; //记录转置后的稀疏矩阵
    for(int i = 1; i <= realnum; i++){
        temp[rowNext[element[i].col]].col = element[i].row;
        temp[rowNext[element[i].col]].row = element[i].col;
        temp[rowNext[element[i].col]].value = element[i].value;
        rowNext[element[i].col] ++;
    }
    int cntrows = rows;
    rows = cols;
    cols = cntrows;
    delete []element;
    delete []colSize;
    delete []rowNext;
    element = temp;
    return;
}


    void OutMatrix(){
        printf("%d",rows);
        printf(" ");
        printf("%d\n",cols);
        int cnt = 1;
        for(int i = 1; i<= rows; i++){
            for(int j = 1; j <= cols; j++){
                if(element[cnt].row == i && element[cnt].col == j){
                    printf("%d ",element[cnt].value);
                    cnt++;
                }
                else{
                    printf("0 ");
                }
            }
            printf("\n");

        }
```

```
    }
    void createMatrix(int newrow, int newcol){
        rows = newrow; cols = newcol;
        element = new matrixTerm[newrow*newcol + 1];
        int t;realnum = t;
        t = Read();
        int x, y, v;
        for(int i = 1; i<=t; i++){
            x = Read(); y = Read(); v = Read();
            element[i].row = x;
            element[i].col = y;
            element[i].value = v;
        }
    }
    int realnum = 0;;//记录稀疏矩阵真实数据个数
    matrixTerm* element = nullptr;
private:
    int rows;
    int cols;
};
int main(){
    int w;
    w = Read();
    int sym;
    spareMatrix *mt = new spareMatrix;
    while(w){
        w--;
        sym    = Read();
        if(sym == 1){
            mt->resetMatrix();
        }
        else if(sym == 2){//进行矩阵乘法
            int nmtrow, nmtcol;
            nmtrow = Read(); nmtcol = Read();
            spareMatrix *newmt = new spareMatrix;
            newmt->createMatrix(nmtrow, nmtcol);
            spareMatrix* temp = new spareMatrix;
            temp->MatrixMultiply(mt,newmt);
            mt = temp;
        }
        else if(sym == 3){//进行矩阵加法
            int nmtrow, nmtcol;
            nmtrow = Read(); nmtcol = Read();
            spareMatrix * newmt = new spareMatrix;
            newmt->createMatrix(nmtrow, nmtcol);
            spareMatrix* temp = new spareMatrix;
```

```
                temp->MatrixPlus(mt, newmt);
                mt = temp;
            }
            else if(sym == 4){
                mt->OutMatrix();
            }
            else if(sym ==5){//进行矩阵转置
                mt->MatrixInverse();
            }
        }
        delete mt;
        return 0;
}
```